A Survey on NFRs for IT Automation Agents

Arjun Vaidya
av3315@columbia.edu
Columbia University
Department of Computer Science
New York, NY, USA

Ayush Bhauwala
ab6106@columbia.edu
Columbia University
Department of Computer Science
New York, NY, USA

Gautam Agarwal
ga2726@columbia.edu
Columbia University
Department of Computer Science
New York, NY, USA

Rishabh Jain
rj2790@columbia.edu
Columbia University
Department of Computer Science
New York, NY, USA

Tanmay Agrawal
ta2832@columbia.edu
Columbia University
Department of Computer Science
New York, NY, USA

Ruchi Mahindru
ruchi.mahindru@ibm.com
IBM Research
Yorktown Heights, NY, USA

ABSTRACT

AI agents are increasingly deployed in IT automation domains such as SRE, CISO operations, and FinOps, yet their evaluation remains dominated by functional success metrics that overlook real-world performance and deployment risks. In practice, agentic systems exhibit non-deterministic reasoning, multi-step tool use, brittle error recovery, and sometimes succeed through unintended paths, revealing a critical gap between benchmark performance and production readiness. This survey synthesizes findings from software engineering, ML system quality models, and recent agentic frameworks to establish a unified understanding of non-functional requirements (NFRs) for AI agents.

CCS CONCEPTS

• **Software and its engineering → General programming languages**.

KEYWORDS

AI agents, non-functional requirements, benchmarking, IT automation, observability, production readiness, performance metrics

ACM Reference Format:
Arjun Vaidya, Ayush Bhauwala, Gautam Agarwal, Rishabh Jain, Tanmay Agrawal, and Ruchi Mahindru. 2026. A Survey on NFRs for IT Automation Agents. In *Proceedings of ACM Conference on Artificial Intelligence Systems (ACM CAIS 2026).* ACM, New York, NY, USA, 8 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Recent progress in LLMs has accelerated the development of AI agents capable of interacting with tools within complex IT environments. Current AI agent benchmarks such as ITBench [7], AgentBench [9], and WebArena [21] predominantly evaluate functional capabilities through task completion rates, revealing significant performance gaps in production scenarios. For instance, ITBench reports success rates of only 11.4% for Site Reliability Engineering (SRE) tasks, 25.2% for Chief Information Security Officer (CISO) operations, and 25.8% for Financial Operations (FinOps) automation. AgentBench reveals that while GPT-4 [14] achieves a high overall score of 4.01 (a weighted average across environments), its performance varies widely across task types, with only 42.4% success on OS tasks, 29.0% on web browsing, and 16.6% on a thinking puzzle game. While these metrics identify functional limitations, they fail to capture the operational characteristics that determine whether agents can operate reliably, efficiently, and cost-effectively at scale in production environments.

Non-Functional Requirements (NFRs), the quality attributes that define how a system operates rather than what it does, remain largely unmeasured in existing agent evaluation frameworks. This gap is critical because systems can satisfy functional requirements yet fail catastrophically on operational metrics such as latency during incident response, cost efficiency at scale, or observability for debugging failures. The distinction between laboratory success and production readiness hinges on comprehensive NFR evaluation across performance, cost, reliability, and observability dimensions.

This survey addresses three key questions: (1) What NFR categories are critical for production-ready AI agents? (2) How do existing benchmarking frameworks address or neglect these requirements? (3) What methodologies and instrumentation strategies enable holistic NFR measurement? We present a comprehensive taxonomy of agent-specific NFRs, analyze current evaluation practices, and identify research opportunities for bridging the production readiness gap.

2 BACKGROUND AND MOTIVATION

AI agents intended for IT automation operate under real-world constraints that differ significantly from the controlled settings used in most functional benchmarks. While the introduction highlights the gap between benchmark success and production reality, understanding why this gap emerges requires examining the architectural foundations of agent systems and the operational context in which they run. Agent design patterns shape how they reason, plan, and execute actions, which in turn creates distinct non-functional trade-offs. Likewise, the domains where these agents are deployed impose

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ACM CAIS 2026, May 26–29, 2026, San Jose, CA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/05
https://doi.org/XXXXXXX.XXXXXXX

specific performance, reliability, cost, and observability expectations that functional metrics alone cannot capture. These factors motivate a closer look at how architecture and domain context interact to determine production readiness, setting the stage for the detailed analysis that follows.

2.1 Agent Architecture and Operational Context

AI agents for IT automation employ diverse architectural patterns, ReAct (Reasoning and Acting) [20], Plan-and-Execute, and Reflection-based [17] approaches, each creating distinct NFR trade-offs. ReAct agents interleave reasoning and action steps, potentially achieving lower latency but higher token consumption through iterative tool calls. Plan-and-Execute architectures separate planning from execution, offering more reliable workflows at the cost of increased planning overhead. These architectural choices directly impact the four critical NFR dimensions: performance (end-to-end latency and throughput), cost efficiency (token consumption and API call frequency), reliability (error rates and graceful degradation), and observability (trace completeness and decision explainability).

2.2 Domain-Specific NFR Priorities

Table 1: Domain-Specific NFR Guidance

Domain	Critical NFR	Threshold	Failure Mode
SRE Response	Latency (TTFT) & Planning Overhead	< 5s (TTFT) < 1 min (Planning)	Financial loss from downtime; Engineers abandon slow agents.
CISO Compliance	Reliability & Trace Completeness	$< 2\%$ False Postive Rate 100% Auditability	Alert fatigue leading to manual override; Regulatory fines.
FinOps	Cost Efficiency & Accuracy	ROI > 1 < 30s Planning	Economic Deficit (Operational costs exceed savings); Forecasting errors.
Customer Support	Latency & Hallucination Rate	< 3s $< 5\%$	Session abandonment; Brand damage.

IT automation spans three distinct operational domains with varying NFR criticality (Table 1). In SRE scenarios, latency requirements are stringent, incident response delays measured in seconds can translate to millions in downtime costs, making time-to-first-token and planning overhead paramount. CISO operations prioritize reliability and auditability, where false positives in threat detection or incomplete compliance traces create regulatory risk. FinOps automation demands accuracy in financial calculations and cost efficiency paradoxically requiring that agent operational expenses be justified by savings generated. Current benchmarks measure domain-specific task success but neglect these contextual NFR thresholds that determine production viability.

2.3 The Production Readiness Gap

The fundamental challenge is that "evaluation without context equals expensive guesswork." Non-contextual evaluations using generic metrics, such as measuring average latency without domain-specific acceptable thresholds, break down for specialized IT workflows. An SRE agent achieving 5-second response time might be acceptable for routine monitoring but catastrophic during live outages, while the same latency proves adequate for FinOps budget analysis. Contextual evaluation requires establishing domain-specific baselines, understanding operational criticality, and instrumenting NFRs within realistic workflow simulations rather than isolated task measurements.

2.4 Case Studies in Agentic NFR Failures

2.4.1 Cross-Repository Data Exfiltration via the GitHub Model Context Protocol (MCP). The vulnerability discovered in the GitHub implementation of the Model Context Protocol (MCP) provides a definitive case study on the risks of cross-domain tool execution[12]. In this incident, an AI agent was successfully manipulated into exfiltrating data from a private repository and publishing it to a public repository after processing a malicious GitHub issue. This event demonstrates a critical failure in Prompt Injection Resilience, where the agent failed to distinguish between system instructions and untrusted data retrieved from an external tool. For IT automation, this incident proves that traditional Role-Based Access Control (RBAC) is insufficient if the agent's logic can be hijacked via data. It necessitates NFRs that enforce strict data-flow isolation and mandate that instructions originating from external environments cannot influence high-privilege tool calls in private environments.

2.4.2 Output Variance in Claude Code and OpenAI Codex Security Audits. The application of LLM-based agents like Claude Code and OpenAI Codex for automated vulnerability detection has highlighted a fundamental challenge in idempotency. Empirical research[3] demonstrated that these agents, when presented with the same codebase and identical prompts across three iterations, failed to produce consistent security findings. Such non-deterministic behavior is incompatible with the requirements of a production CI/CD pipeline, where a security gate must provide a repeatable "pass" or "fail" signal. This incident underscores the necessity of NFRs that quantify and limit output variance. Without guaranteed idempotency, AI agents introduce "flaky" infrastructure risks, where the presence or absence of a security alert is a factor of probabilistic sampling rather than a stable assessment of the system's state.

These incidents serve as empirical evidence that the categories defined in the taxonomy below are not merely theoretical considerations but essential guardrails for production stability. By mapping these real-world vulnerabilities to specific NFR constraints, organizations can move from reactive troubleshooting to a proactive, requirement-driven approach for deploying autonomous IT agents.

2.5 Validity and Rigor in Agentic Benchmarks

Recent scholarship has increasingly questioned the reliability of standard functional benchmarks, arguing that high pass rates often mask critical validity flaws. In a comprehensive audit of ten widely used agentic benchmarks, [22] introduced the Agentic Benchmark Checklist (ABC) to evaluate outcome validity, task validity, and reporting rigor. Their analysis revealed pervasive structural deficiencies, such as "do-nothing" agents achieving inflated scores on τ-Bench through gamification and invalid success metrics in WebArena that rely on superficial string matching.

Most significantly, the study provided empirical quantification of data contamination issues in code generation tasks. By comparing

Table 2: Using NFRs to Detect Benchmark Validity Issues, Kang et.al.[22]

Validity Failure	Operational Manifestation	NFR Diagnostic (Detection Strategy)
The "Do-Nothing" Agent(*Task Validity*)	In τ-bench [19], agents achieved 38% success on impossible tasks (e.g., refunding a non-refundable ticket) simply by returning empty responses.	**Workflow Path Efficiency:** A task flagged as *Success=True* but having a *Step Count=0* or *Tool Calls=0* is immediately rejected as a false positive.
Solution Leakage (*Task Validity*)	In SWE-bench [8], 32.67% of issues leaked solution patches in the problem description, allowing agents to copy-paste rather than code.	**Planning Overhead:** Agents solving complex engineering tasks with near-zero reasoning latency (< 100ms) or skipping the *Context Retrieval* phase indicate leakage.
Environment Drift (*Task Validity*)	OSWorld [18] performance was underestimated by 28% because website selectors changed, causing capable agents to fail.	**Error Rate by Type:** A statistical spike in *ElementNotFoundError* (Infrastructure Error) versus *LogicError* (Agent Error) isolates environment drift from model incompetence.
Unreliable Judges (*Outcome Validity*)	WebArena's [21] LLM-as-a-judge accepted "45+8≠63" as correct, inflating scores for weak models.	**Auditability:** Tracing the *Judge's* own reasoning trace (meta-evaluation) allows auditors to validate the scoring logic, not just the agent's output.

performance on the public SWE-bench Verified [8] against SWE-bench Pro - a private dataset constructed from commercial codebases distinct from public training corpora - the authors observed a precipitous performance degradation from > 70% to ∼ 23%. This discrepancy underscores that current functional metrics frequently measure training data memorization rather than generalized reasoning capabilities.

These findings highlight the necessity of moving beyond simple pass/fail functional metrics toward more diagnostic frameworks which include Non-Functional Requirements (NFRs) in evaluations. Our NFR taxonomy serves as a necessary diagnostic layer to detect these validity failures. By measuring process-oriented NFRs alongside functional success, we can distinguish genuine capability from benchmark gaming. Table 2 illustrates how specific NFR metrics expose common validity threats identified in the ABC audit.

3 RELATED WORK AND EXISTING METRICS

This section reviews the evolution of NFR frameworks, moving from static software quality models to the dynamic, observable behaviors required for Agentic AI.

3.1 From Software Quality to ML Robustness

The foundational standard for NFR metrics is ISO/IEC 25010, which establishes eight broad quality categories including Performance, Reliability, and Security [10]. As systems evolved to incorporate Machine Learning, these frameworks were extended by ISO/IEC 25059 [5] and research identifying 34 ML-specific characteristics, such as data suitability, fairness, and robustness against noise [13, 4]. However, these models primarily address *prediction quality* (e.g., accuracy, F1-score) rather than the *process quality* of autonomous agents. Bibliometric analysis confirms that while NFR prioritization is domain-dependent [1], existing taxonomies fail to capture the multi-step reasoning, tool invocation, and state management that define agentic workflows.

3.2 Agent-Specific Observability

To address the opacity of autonomous systems, recent research has shifted from static evaluation to runtime observability. Dong et al. introduced the **AgentOps** framework, which argues that agents require tracing unique artifacts distinct from traditional software [2]. Specifically, they define four observability dimensions: *Reasoning* (chain-of-thought traces), *Planning* (task decomposition graphs), *Workflows* (multi-step execution paths), and *Guardrails* (safety policy adherence). Unlike traditional APM tools that track simple request-response latency, AgentOps emphasizes capturing the intermediate "thought" states necessary to debug non-deterministic failures.

Complementing these academic frameworks, industry standardization is coalescing around the OpenTelemetry (OTel) GenAI Semantic Conventions [15]. This initiative aims to unify telemetry across fragmented frameworks (e.g., LangChain, AutoGPT) by defining shared schemas for attributes such as `gen_ai.system`, `gen_ai.tool.execution`, and token usage buckets. While OTel provides the *structural* standards for collecting data, it does not define the *evaluative* thresholds, leaving a gap between raw telemetry and actionable quality assessments.

3.3 Benchmarking Reliability and NFRs

Recent benchmarking efforts have highlighted the critical gap between functional competence and production reliability. AgentBench [9] and ToolLLM [16] provide comprehensive environments for evaluating reasoning and tool-use capabilities, yet they primarily report functional success rates (Pass@1) without detailed profiling of latency, cost, or varying context window consumption.

Addressing this reliability gap, Sierra's τ-bench specifically targets the fragility of agents in dynamic environments [19]. By simulating multi-turn, stateful interactions, they utilize the *pass@k* metric to quantify consistency. Their findings—that state-of-the-art models drop to below 25% consistency (*pass*@8) in realistic domains—provide empirical validation for the *Idempotency* and *Regression Resistance* attributes proposed in our taxonomy.

CLEAR framework [11] introduces a five-dimensional model for enterprise LLM evaluation: Cost (cost-normalized accuracy), Latency (SLA compliance), Efficacy (task quality), Assurance (policy adherence), and Reliability (pass@k). An analysis of 12 widely used benchmarks reveals substantial validity flaws, 8 of 10 benchmarks exhibit severe cost misestimation (up to 100%) and extreme economic variance, with agents of comparable accuracy differing by up to 50× in cost ($0.10 vs. $5.00 per task). While our taxonomy aligns with CLEAR at the dimension level, we extend it by operationalizing these axes into fine-grained Level-2 metrics, bridging high-level enterprise objectives with low-level agent instrumentation.

4 NFR TAXONOMY

Existing NFR frameworks such as ISO/IEC 25010, ISO/IEC 5338 [6], and the AI-focused ISO/IEC 25059 [5] provide well established quality models for traditional software and for machine learning systems. These models are valuable foundations, although they do not fully account for the operational characteristics of AI agents. Unlike static models, agents perform multi step reasoning, interact with external tools in real time, maintain long running state, and display nondeterministic planning behavior. These properties introduce distinct failure modes and quality considerations that require an expanded, agent centred view of NFRs.

Recent work on ML system quality highlights important dimensions such as robustness, data quality, fairness, and model reliability. While these dimensions remain relevant, they do not cover the full range of behaviors that emerge in agentic workflows. Agents introduce additional layers of complexity such as variability in workflow paths, dependency on tool calls, management of context windows, and emergent reasoning artifacts. These characteristics fall outside the scope of existing taxonomies, which means that classical NFR categories must be adapted and extended to reflect the realities of agent based execution.

To close this gap, we propose a two level NFR taxonomy designed specifically for AI agents. Level 1 defines broad quality categories that remain aligned with established software engineering standards. Level 2 defines detailed, agent specific metrics that reflect concrete behaviors observed during planning, reasoning, and execution. This structure preserves compatibility with existing quality models while enabling more granular and operationally meaningful evaluation of agentic capabilities in production environments.

Table 1 summarizes the taxonomy. It outlines each Level 1 category, the associated Level 2 metrics, and the operational definitions required for practical measurement and assessment.

5 MEASUREMENT METHODOLOGIES

This section outlines practical methods for evaluating the performance and reliability of agentic systems. Standard metrics focus on final outputs, yet agents operate through multi step reasoning and tool use, which requires more granular assessment. We introduce approaches that capture both functional outcomes and the underlying behaviors that drive them. The subsections contrast contextual and non contextual evaluation, describe instrumentation techniques for collecting detailed execution data, and present scoring models that merge functional and non functional signals into deployment criteria. Together, these methods provide a structured basis for judging agent readiness in real operational settings.

5.1 Contextual vs. Non-Contextual Evaluation

Non-contextual evaluations assess agents using generic, standardized metrics that ignore domain-specific requirements and enterprise context. These approaches typically employ off-the-shelf benchmarks measuring general capabilities without considering whether agents correctly interpret internal documentation, proprietary policies, or company-specific constraints. For example, generic language understanding benchmarks cannot assess whether a CISO agent properly applies organization-specific security policies or whether an SRE agent navigates proprietary monitoring tools effectively. The limitations manifest as surface-level measurements that don't drill into workflow stages or decision-making processes. Measuring only final output accuracy without understanding where failures occur, at planning, tool selection, or execution stages, makes targeted optimization nearly impossible. Generic NFR metrics measured in isolation, such as average latency or total cost, fail to establish meaningful thresholds; acceptable performance varies dramatically between use cases even within the same domain. Contextual evaluation, in contrast, custom-builds assessments around each agent's specific purpose, domain constraints, and organizational context. This requires systems-thinking that ties measurements to enterprise workflows and real operational criticality. For IT automation agents, contextual evaluation establishes domain-specific NFR baselines: SRE incident response requiring sub-5-second latency versus FinOps budget analysis tolerating 30-second planning overhead. Effective contextual approaches weight NFRs according to operational impact, prioritizing observability for post-incident analysis in SRE while emphasizing auditability for CISO compliance scenarios.

5.2 Instrumentation Techniques

Capturing comprehensive NFRs requires telemetry collection at multiple granularities. Trace-based approaches log complete execution paths including reasoning steps, tool invocations with parameters, intermediate outputs, and decision points. Frameworks like AgentOps and OpenTelemetry GenAI conventions provide standardized instrumentation for latency tracking, token counting, and error classification. Metrics-based systems aggregate statistics across executions, average latency distributions, token consumption percentiles, and error rate trends, enabling comparative analysis between agent architectures or optimization strategies. Component-level breakdown instruments individual workflow stages: planning accuracy (does task decomposition align with goals?), tool selection correctness (right API with proper parameters?), and intermediate output validation (e.g., valid SQL queries in FinOps). Path efficiency analysis evaluates whether agents take optimal routes or inefficient detours, measuring step count, redundant tool calls, and reasoning

Table 3: AI Agent NFR Taxonomy (Level 1 & Level 2 Attributes)

Level 1 Category	Level 2 Metric	Definition
Performance	End-to-end latency	Total time from task initiation to completion.
	Planning overhead	Time spent in reasoning cycles, reflection loops, or plan construction.
	Tool call latency	Delay between issuing a tool request and receiving a response.
	Time-to-first-token	Time until the first token is generated after the request is sent.
	Throughput under load	Number of tasks the agent can process concurrently without degradation.
	Workflow path efficiency	Degree to which the agent follows minimal, intended steps (avoids detours).
	Context window utilization	How effectively the agent uses available context to avoid truncation or loss.
Cost Efficiency	Token consumption per task	Average number of tokens used to complete a task.
	API call count	Total number of tool/LLM calls invoked per workflow.
	Cost per successful/failed task	Monetary cost associated with completed and failed attempts.
	Retry-induced overhead	Additional cost caused by agent retries or error recovery loops.
Reliability	Task success rate	Percentage of tasks completed correctly.
	Error rate by type	Frequency of tool errors, reasoning errors, hallucinations, etc.
	Tool selection accuracy	Precision of selecting the correct tool for a specific subtask.
	Error recovery efficiency	Success rate of self-correction attempts following a failure.
	Retry frequency	Number of retries required to complete a task.
	Idempotency	Whether repeated executions yield consistent, stable results.
	Regression resistance	Stability of behavior over time when fixes/updates are introduced.
	Distribution shift robustness	Performance stability on out-of-distribution or novel inputs.
	Uncertainty calibration	Correlation between agent's confidence scores and actual correctness.
	Coordination failure rate	Frequency of inter-agent deadlocks, loops, or consensus failures.
Explainability	Trace completeness	Availability of plans, reasoning summaries, and tool I/O.
	Decision explainability	Clarity of why specific actions or tool calls were chosen.
	Workflow milestone visibility	Ability to observe intermediate stages of execution.
	Guardrail/policy adherence visibility	Whether the agent exposes safety/guardrail constraints in outputs.
	OpenTelemetry compatibility	Support for standardized tracing and observability frameworks.
Scalability	Concurrency capacity	Maximum parallel workflows without failure.
	Multi-agent coordination efficiency	Performance changes when multiple agents collaborate.
	Cross-domain scaling	Ability to maintain quality across different task domains.
	Inter-agent comm. overhead	Ratio of communication tokens/time vs. task execution tokens/time.
	Negotiation latency	Time consumed by agents to reach consensus before acting.
	Swarm coherence	Stability of collective behavior (absence of cascading feedback loops).
	Resource contention rate	Frequency of conflicting tool/resource access attempts by multiple agents.
Security	Access control adherence	Correct compliance with permission and role constraints.
	Data sanitization	Avoiding leakage of sensitive information to external models.
	Auditability	Ability to trace decisions, tool actions, and security-relevant events.
	Supply chain integrity	Verification status of third-party tools, plugins, and model dependencies.
Robustness	Single-turn attack resilience rate	Fraction of adversarial single-turn inputs that do not induce policy-violating actions or unsafe tool calls.
	Multi-turn manipulation resistance	Resistance to gradual prompt steering across multiple turns (policy adherence maintained over an interaction).
	Cross-modal input sanitization	Effectiveness of sanitizing/isolating untrusted non-text inputs (e.g., OCR from images, transcripts) before they influence decisions.
	Indirect injection detection rate	Detection rate of prompt injection embedded in tool outputs (e.g., web pages, logs, tickets) and other retrieved context.
	Guardrail bypass resistance	Rate at which the agent avoids violating constraints under adaptive bypass attempts (jailbreak-style probing).
Usability	Human intervention frequency	How often humans must step in to correct or guide the agent.
	Time-to-correct	Time required for a human to correct or override the agent.
	Output interpretability	How easily humans understand the agent's outputs.

overhead, crucial since unintended paths may succeed functionally but fail on latency or cost NFRs.

5.3 Alignment with OpenTelemetry Standards

To enable standardized instrumentation, we map our proposed NFR taxonomy to the emerging OpenTelemetry (OTel) GenAI Semantic Conventions [15]. Currently in active development/experimental status, the OTel GenAI initiative defines attributes for spans such as `gen_ai.system` and `gen_ai.request.model`, alongside token metrics with explicit bucket boundaries.

However, current conventions focus primarily on LLM inference rather than agentic workflows. Proposed extensions aim to introduce concepts such as *Tasks* (minimal trackable units), *Actions* (execution methods), and *Memory* (persistent context). Our taxonomy complements these efforts by defining the *quality semantics* for these structural components. Table 4 illustrates the mapping between our Level 2 metrics and OTel attributes.

Table 4: Mapping NFR Metrics to OpenTelemetry Attributes

Taxonomy Metric	OpenTelemetry Attribute / Span Event
End-to-end Latency	`duration_ms` on `invoke_agent` span
Token Consumption	`gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`
Tool Call Latency	`duration_ms` on `gen_ai.tool.execution` span
Error Rate by Type	`status.code` = `ERROR`, `error.type` (e.g., *ToolError*, *ContextLimit*)
Trace Completeness	`gen_ai.content.prompt` + `gen_ai.content.completion` (ensuring `capture_content=true`)
Cost per Task	Derived metric: ∑(`tokens` × `cost_factor`) aggregated by `trace_id`
Evaluation Score	`gen_ai.evaluation.result` (custom event for scoring pass/fail/score)

By adopting these semantic conventions, organizations can leverage existing observability pipelines (e.g., Jaeger, Prometheus) to visualize agent NFRs alongside traditional microservices telemetry.

5.4 Tooling Landscape & Gap Analysis

To assess the practical feasibility of measuring our proposed NFRs, we analyzed the capabilities of five leading agent observability platforms. Table 5 maps their native features to our NFR categories. Note that while basic Performance and Cost metrics have become commoditized (supported by most tools), significant divergence exists in how Reliability and Security are addressed.
Capability Analysis:

- **Performance & Cost (Commoditized):** Basic metrics like *Latency* and *Token Consumption* are provided by most tools like *Langfuse*, *Arize*, and *Traceloop*.
- **Reliability (Bifurcated Approach):** There is a split between *active* and *passive* reliability.
  - *Active:* **PortKey** (Gateway) enforces reliability via retries, fallbacks, and load balancing at the network layer.
  - *Passive:* **Arize Phoenix** monitors reliability via semantic drift detection and evaluation scores.
- **Security (Emerging):** Most observability tools treat security merely as logging. **PortKey** (via Gateway Guardrails) and **AgentOps** (via PII/Secret detection) are exceptions that natively map to our *Security* NFRs, actively flagging or blocking violations like prompt injection.

This landscape indicates that while standard metrics are well-covered, *Scalability* remains a major gap, with no single tool explicitly measuring concurrency limits or multi-agent coordination overhead natively.

5.5 Production-Readiness Scoring

Multi-dimensional scoring models combine functional success with weighted NFR performance to assess deployment viability. A production ready agent must not only complete tasks (functional requirement) but do so within acceptable latency bounds, cost constraints, error tolerances, and observability standards (non-functional requirements). Scoring frameworks establish domain-specific thresholds for each NFR category, then compute composite scores that penalize violations. For instance, an SRE agent achieving 90% task success but exceeding latency thresholds 40% of the time receives degraded production-readiness scores despite functional competence. Holistic scoring addresses the challenge that "agents can use different paths to succeed; not all of them are as intended." Rubric-based evaluation using LLM-as-judge assesses whether successful paths followed safe, compliant routes or achieved results through unintended shortcuts (e.g., bypassing security checks in CISO tasks). This prevents rewarding agents that succeed via methods unsuitable for production deployment despite achieving correct final outputs.

To formalize the assessment within our proposed taxonomy, we introduce the *Agent Readiness Score* (ARS). This metric applies a strict binary feasibility constraint when critical Level 1 thresholds are violated, and otherwise quantifies performance using a weighted aggregation of Level 2 metrics:

$$ARS = \begin{cases} 0 & \text{if } \exists c \in \mathcal{G} \text{ such that } S_c < \tau_c \\ \sum_{c \in C} \alpha_c S_c & \text{otherwise} \end{cases} \quad (1)$$

$$S_c = \sum_{a \in \mathcal{A}_c} w_a \cdot v_a \quad (2)$$

Where S_c represents the agent's performance in a specific Level 1 category c, and α_c denotes the predefined importance weight for that category. The term τ_c signifies the minimum acceptable score (threshold) for category c within a specific domain context, and $\mathcal{G}$ represents the set of mandatory gatekeeper categories. The category-level score S_c is derived from the underlying Level 2 attributes, where $\mathcal{A}_c$ is the set of attributes belonging to category

Table 5: Comparison of Observability Tools against NFR Categories

Tool	Primary Focus	Performance	Cost	Reliability	Explainability	Security
Langfuse	Open-source tracing & prompt management	●	●	○	●	○
PortKey	AI Gateway (Routing, Fallbacks)	●	●	●	○	●
Arize Phoenix	Evaluation & Drift Detection	●	●	●	●	○
Traceloop	OpenLLMetry (OTel-native instrumentation)	●	●	○	●	○
AgentOps	Multi-Agent Session Replays	●	●	○	●	●

● = Native Support (e.g., built-in dashboards, specific metrics). ○ = Partial/Passive Support (requires custom config).

c, v_a is the normalized value of attribute a, and w_a is its relative importance weight.

Attribute values v_a are derived using a two-tiered methodology that combines objective telemetry with qualitative assessment of reasoning quality, resulting in two evaluation options.

- **Telemetry-based Evaluation:** Attributes with deterministic values (e.g., latency, token consumption, and error rates), they are directly computed from execution logs and telemetry traces.
- **LLM-based Evaluation:** For complex attributes not fully captured by deterministic metrics (e.g. decision explainability or guardrail adherence), an LLM-based judge performs a review of the execution trace, including reasoning steps and tool I/O, and assigns a normalized score.

6 GAPS AND RESEARCH OPPORTUNITIES

Despite advancements in observability tools and the existence of general software quality standards (such as ISO 25010) and AI-specific extensions (ISO 25059), significant gaps remain in the practical benchmarking of AI Agents. The transition from static ML models to autonomous agents has outpaced the development of rigorous evaluation frameworks. Consequently, existing approaches often fail to address the complex, non-deterministic nature of agentic workflows. This section identifies critical limitations in current approaches and outlines key directions for future research.

6.1 Standardization and Benchmarking Frameworks

A primary limitation is the lack of standardized NFR benchmarking frameworks. While benchmarks such as ITBench [7], AgentBench [9], and WebArena [21] provide domain-specific functional evaluations, they lack instrumentation for essential NFRs like latency profiling, token tracking, and error classification. This absence prevents the systematic evaluation of architectural trade-offs and optimization strategies. There is an urgent need to develop standardized telemetry collection protocols compatible with diverse frameworks (e.g., LangChain, AutoGPT). Future work should focus on establishing baseline NFR profiles for common architectures and creating community-driven reference datasets with ground-truth NFR annotations. This would enable cross-framework comparisons and reproducible research without exposing proprietary enterprise information.

6.2 Holistic and Multi-Dimensional Evaluation

Current methodologies predominantly measure NFR dimensions in isolation, failing to capture their interdependencies. For instance, an agent optimized solely for cost might compromise reliability via excessive tool call reduction, while latency optimization could inadvertently spike token consumption through parallel execution. Future frameworks must adopt multi-objective optimization approaches that reveal Pareto frontiers: configurations where improving one metric necessitates the degradation of another. Furthermore, the development of synthetic data generation techniques offers a pathway to create realistic automation scenarios with controlled NFR characteristics (e.g., varying API latencies and failure modes). This allows for rigorous stress testing without relying on sensitive production data.

6.3 Process Quality and Safety Validation

Agents operate under critical assumptions regarding non-determinism, reasoning soundness, and context management that are rarely validated explicitly. A significant risk involves agents "succeeding via unintended paths," where correct outcomes are achieved through unsafe shortcuts, redundant tool calls, or flawed logic that may not generalize. Evaluation protocols must evolve to be path-aware, scoring process quality alongside final outcomes. Research should focus on component-level rubrics that assess planning coherence and intermediate output validity. Additionally, frameworks are needed to test assumption boundaries through adversarial inputs and dynamic environments, ensuring agents remain robust against edge cases and context management failures.

6.4 Adaptive Optimization and Generalization

Manual NFR optimization, such as static adjustments to context windows or retry policies, does not generalize well across diverse tasks or varying deployment environments. There is a significant opportunity for auto-tuning frameworks that leverage production telemetry to learn optimal configurations dynamically. Reinforcement learning approaches could optimize agent policies directly for NFR objectives, balancing task success with latency and cost constraints. Furthermore, hierarchical scheduling systems could be developed to route tasks to agent instances with appropriate resource allocations based on real-time system states (e.g., differentiating between peak and off-peak traffic requirements).

7 CONCLUSION

AI agents are rapidly transitioning from research prototypes to operational components of IT automation pipelines, yet their evaluation remains dominated by functional success metrics that fail to capture the complexities of real-world deployment. This survey demonstrates that existing software and AI quality frameworks, while comprehensive for traditional systems, do not fully account for the nondeterministic reasoning, dynamic tool use, and workflow variability that define agentic behavior. To bridge this gap, we introduced an agent-centric NFR taxonomy that extends established standards with fine-grained metrics reflective of planning quality, tool-call dynamics, context management, observability, and safety.

Our analysis of current benchmarks reveals that functional metrics alone are insufficient indicators of production readiness. The absence of standardized telemetry, inconsistent observability practices, and the lack of multi-dimensional evaluation methodologies prevent meaningful comparison across agent architectures and deployment contexts. The NFR taxonomy and measurement methodologies outlined in this paper provide a foundation for structured, holistic assessment, enabling organizations to reason about latency, cost, reliability, and explainability within domain-specific operational constraints.

Looking ahead, significant research opportunities remain. Standardized NFR benchmarking frameworks, path-aware evaluation methods, and adaptive optimization systems are essential for the next generation of production-ready agents. As agentic systems continue to integrate into mission-critical workflows, robust NFR measurement and optimization will become as important as functional accuracy. Establishing unified, empirically grounded NFR standards will be pivotal in enabling reliable, efficient, and auditable agent deployments at scale.

ACKNOWLEDGMENTS

We would like to acknowledge that this work is the result of a collaborative team research effort. We extend our sincere gratitude to our supervisor, Ruchi Mahindru, for her guidance, feedback, and support throughout the development of this paper. We also thank the supporting resources, tools, and academic infrastructure that facilitated our research and analysis.

REFERENCES

[1] Vincenzo De Martino and Fabio Palomba. 2025. Classification and challenges of non-functional requirements in ml-enabled systems: a systematic literature review. *Information and Software Technology*, 181, 107678. DOI: https://doi.org/10.1016/j.infsof.2025.107678.

[2] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. Agentops: enabling observability of llm agents. (2024). https://arxiv.org/abs/2411.05285 arXiv: 2411.05285 [cs.AI].

[3] Romain Gaucher, Vasilii Ermilov, and Clint Gibler. 2025. Finding vulnerabilities in modern web apps using Claude Code and OpenAI Codex. Empirical study on non-determinism in AI-driven security auditing. (2025). https://semgrep.dev/blog/2025/finding-vulnerabilities-in-modern-web-apps-using-claude-code-and-openai-codex/.

[4] Khan Mohammad Habibullah, Gregory Gay, and Jennifer Horkoff. 2023. Non-functional requirements for machine learning: understanding current use and challenges among practitioners. *Requirements Engineering*, 28, 2, 283–316. ISBN: 1432-010X. DOI: 10.1007/s00766-022-00395-3.

[5] International Organization for Standardization. 2023. ISO/IEC 25059:2023 – software engineering – systems and software quality requirements and evaluation (SQuaRE) – quality model for AI systems. (2023). https://www.iso.org/standard/80655.html.

[6] International Organization for Standardization. 2023. ISO/IEC 5338:2023 – information technology – artificial intelligence – AI system life cycle processes. (2023). https://www.iso.org/standard/81118.html.

[7] Saurabh Jha, Rohan R. Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, et al. 2025. Itbench: evaluating ai agents across diverse real-world it automation tasks. *arXiv preprint arXiv:2502.05352*. https://arxiv.org/abs/2502.05352.

[8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*. https://arxiv.org/abs/2310.06770.

[9] Xiao Liu et al. 2023. Agentbench: evaluating llms as agents. *arXiv preprint arXiv:2308.03688*.

[10] Dewi Mairiza, Didar Zowghi, and Nurie Nurmuliani. 2010. An investigation into the notion of non-functional requirements. In *Proceedings of the 2010 ACM Symposium on Applied Computing* (SAC '10). Association for Computing Machinery, Sierre, Switzerland, 311–317. ISBN: 9781605586397. DOI: 10.1145/1774088.1774153.

[11] Sushant Mehta. 2025. Beyond accuracy: a multi-dimensional framework for evaluating enterprise agentic ai systems. *arXiv preprint arXiv:2511.14136*. https://arxiv.org/abs/2511.14136.

[12] Marco Milanta and Luca Beurer-Kellner. 2025. GitHub MCP exploited: accessing private repositories via MCP. Technical blog post on prompt injection in AI agent protocols. (2025). https://invariantlabs.ai/blog/mcp-github-vulnerability.

[13] Koji Nakamichi, Kyoko Ohashi, Isao Namba, Rieko Yamamoto, Mikio Aoyama, Lisa Joeckel, Julien Siebert, and Jens Heidrich. 2020. Requirements-driven method to determine quality characteristics and measurements for machine learning software and its evaluation. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*, 260–270. DOI: 10.1109/RE48521.2020.00036.

[14] OpenAI. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*. https://arxiv.org/abs/2303.08774.

[15] OpenTelemetry Authors. 2024. Semantic conventions for generative AI systems. OpenTelemetry Semantic Conventions, Experimental. (2024). https://opentelemetry.io/docs/specs/semconv/gen-ai/.

[16] Yujia Qin et al. 2023. Toolllm: facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.

[17] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*. https://arxiv.org/abs/2303.11366.

[18] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, et al. 2024. Osworld: benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*. https://arxiv.org/abs/2404.07972.

[19] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: a benchmark for tool-agent-user interaction in real-world domains. (2024). https://arxiv.org/abs/2406.12045 arXiv: 2406.12045 [cs.AI].

[20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*. https://arxiv.org/abs/2210.03629.

[21] Shuyan Zhou et al. 2023. Webarena: a realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*. https://arxiv.org/abs/2307.13854.

[22] Yuxuan Zhu, Tengjun Jin, Daniel Kang, et al. 2025. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*. https://arxiv.org/abs/2507.02825.